

ST-LEGO: Large Language Models as Modular Architects for Traffic Prediction

Shuhao Li
Fudan University
Shanghai Key Laboratory
of Data Science
Shanghai, China
shli23@m.fudan.edu.cn

Weidong Yang*
Fudan University
Shanghai, China
Zhuhai Fudan Innovation
Research Institute
Zhuhai, China
wdyang@fudan.edu.cn

Yue Cui
Tongyi Laboratory
Alibaba Group
Hangzhou, China
cuipaofu@gmail.com

Lipeng Ma
Fudan University
Shanghai Key Laboratory
of Data Science
Shanghai, China
lpma21@m.fudan.edu.cn

Yixuan Li
Fudan University
Shanghai Key Laboratory
of Data Science
Shanghai, China
yxli24@m.fudan.edu.cn

Chaoteng Wu
Fudan University
Shanghai SEARI Intelligent
System Co., Ltd.
Shanghai, China
ctwu21@m.fudan.edu.cn

Lu Qin
University of Technology
Sydney
Sydney, Australia
lu.qin@uts.edu.au

Fan Zhang
Guangzhou University
GZHU-SCHB Intelligent
Transportation Joint Lab
Guangzhou, China
zhangf@gzhu.edu.cn

Abstract

Traffic prediction serves as a cornerstone for systems and network services such as the Web of Vehicles (WoV), online navigation, and smart city applications. Despite the proliferation of model architectures in recent years, existing approaches often suffer from highly customized structures and weak transferability, making it difficult to cope with increasing task heterogeneity and modeling complexity. To address these challenges, we propose **ST-LEGO**, a modular assembly framework driven by large language models (LLMs) that supports flexible structural composition and automated code generation. ST-LEGO employs a multi-agent collaborative system comprising a Prompt Agent, Assemble Agent, and Code Agent, which are responsible for understanding task requirements, dynamically assembling structural modules, and automatically generating executable PyTorch code. By introducing a standardized module library and an intermediate structural description language (DSL), the framework enables controllable generation, reusable composition, and cross-task generalization of model architectures. Empirical results on multiple real-world traffic datasets demonstrate that models generated by ST-LEGO achieve superior accuracy, structural diversity, and convergence compared to a wide range of manually designed baselines. These results highlight the unique potential and scalability of LLMs as structural architects for traffic prediction, offering a new paradigm for integrating language models into web-interactive intelligent transportation systems.

CCS Concepts

• **Computing methodologies** → **Multi-agent planning**.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792132>

Keywords

Neural Network Generation, Code Generation, Traffic Prediction, Multi-Agent Systems

ACM Reference Format:

Shuhao Li, Weidong Yang, Yue Cui, Lipeng Ma, Yixuan Li, Chaoteng Wu, Lu Qin, and Fan Zhang. 2026. ST-LEGO: Large Language Models as Modular Architects for Traffic Prediction. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792132>

1 Introduction

With the rapid development of autonomous vehicles, the Web of Vehicles (WoV), smart city initiatives, and web-based traffic management systems such as online scheduling platforms and traffic signal control, the importance of traffic prediction has become increasingly prominent [23, 25]. Recent years have witnessed substantial progress driven by deep learning models based on graph neural networks [19, 52, 53], sequence models including recurrent neural networks and temporal convolutional networks [6], as well as attention-based architectures [24, 44]. These models have led to significant advances in architectural design but often remain tightly coupled to specific datasets and scenarios, exhibiting high structural customization and poor transferability. More specifically, different model architectures exhibit notable differences in how they encode spatial dependencies (e.g., graph convolution vs. spatial attention) [31, 58], model temporal dynamics (e.g., GRU vs. Transformer) [2, 44], and handle heterogeneous information (e.g., multi-view graphs, external factors) [9, 13, 30]. These structures are typically "hard-coded" in specific implementations, making them difficult to flexibly replace or combine. Moreover, researchers frequently need to manually design or adjust model architectures for different application requirements, where varying demands on efficiency and accuracy often require entirely different encoding pipelines. This results in poor model reusability and high development costs. The lack of a systematic mechanism for managing and composing model modules further limits the feasibility of large-scale automated modeling.

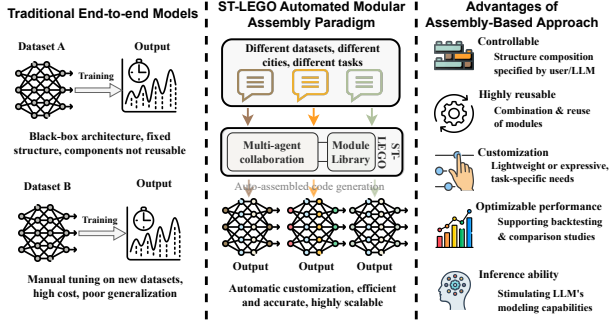


Figure 1: Comparison and advantage analysis of the assembly paradigm vs. traditional end-to-end models.

At the same time, as illustrated in **Figure 1**, traffic prediction tasks themselves are exhibiting increasing task heterogeneity, such as differences in spatial distribution and scale across datasets and cities [59]. Faced with such a highly variable and ever-expanding task space, the traditional "one-model-per-task" approach is becoming increasingly inadequate. While one-for-all models trained on large-scale data promise better generalization [11, 57], they impose stringent requirements on data quality and computational resources. Thus, there is an urgent need for more flexible, composable, and generalizable modeling mechanisms.

In parallel, LLMs have demonstrated strong generalization across diverse tasks, including code generation, decision-making, and modeling of tabular and graph structures [42, 51, 60]. This makes LLMs naturally complementary to the growing task heterogeneity in traffic prediction. However, model development in this domain remains largely manual, involving architecture selection, module configuration, and code customization to meet task-specific needs. Given the abundance of existing components such as GCN, GRU, WaveNet, and Diffusion Convolution, a natural question arises: **Can we leverage the language understanding and compositional capabilities of LLMs to modularize spatio-temporal modeling and enable language-driven model assembly?**

Despite the appeal of "LLM-based automatic assembly of traffic prediction models," as summarized in **Figure 1**, is evident, achieving this goal still faces several key challenges:

First, **traffic prediction on graph-structured data presents heterogeneous task characteristics**, ranging from efficiency constraints and accuracy requirements to the availability of adjacency information. These variations demand different spatial granularities, temporal horizons, and feature integrations, posing the challenge of enabling LLMs to accurately interpret task needs and align them with appropriate modular capabilities. Second, **constructing high-performing forecasting models requires the coordinated use of input encoders, spatial and temporal operators, and fusion mechanisms**, which in turn calls for precise module specifications, a standardized and reusable library, and principled prompting strategies to guide LLMs in planning coherent structures. Without such design discipline, modular assembly risks ad-hoc stacking with weak generalization. Third, **current LLMs still exhibit variability and bias in structure generation and instruction following** [54], making it critical to ensure consistent and reliable assembly, prevent incompatible or ill-posed combinations, and maintain convergence and stability when translating natural language descriptions into executable forecasting architectures.

To address these challenges, we propose ST-LEGO, a multi-agent framework driven by LLMs to assemble diverse graph-based traffic prediction models using modular components. The framework aims to simplify model construction, improve generalization and adaptability, and explore the potential of LLMs in structural reasoning and system design for non-language tasks through a language-and-module paradigm. We design three collaborative agents for ST-LEGO: the Prompt Agent, which interprets user requirements and extracts dataset characteristics; the Assemble Agent, which composes structural modules into model architectures; and the Code Agent, which translates assembled structures into executable PyTorch code. We further construct a standardized module library covering mainstream graph-based spatio-temporal modeling approaches—including spatial modules such as GCN and DiffusionGCN, temporal modules such as GRU, TCN, and Transformer, and fusion strategies such as addition and concatenation—each documented with unified input/output interfaces and functional semantics to support flexible composition. In addition, we design a benchmark repository of historically effective structures and introduce mutation strategies to guide the Assemble Agent, enabling it to identify suitable architectures while avoiding local optima. To ensure transparency and robustness, we also propose an interpretable intermediate structural description language (DSL) that bridges LLM-generated blueprints with executable implementations. Together, these designs establish a new paradigm where language drives rapid model assembly while modular design ensures reliability and adaptability in traffic prediction tasks.

In summary, our main contributions are as follows:

- We propose a new paradigm for traffic prediction using large language models, treating the LLM as a structure constructor that guides model code generation through natural language interaction.
- We develop a multi-agent system—consisting of Prompt, Assemble, and Code Agents—that automates the entire process from requirement interpretation to structure generation and executable code.
- We construct a module library of mainstream graph-based spatio-temporal components, supporting both LLM-driven and user-defined model composition.
- ST-LEGO leverages a benchmark of historically effective structures and mutation strategies to explore optimal architectures for traffic prediction. Its automatically assembled models outperform existing baselines on multiple real-world datasets, demonstrating strong generalization.

2 Preliminaries

Problem Definition. Traffic prediction aims to estimate future traffic states across multiple spatial nodes based on historical observations. Unlike standard time series forecasting, it requires capturing both spatial dependencies among nodes and temporal dynamics at each node. Formally, given an input sequence:

$$\mathbf{X} = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_T\} \in \mathbb{R}^{T \times N \times F}, \quad (1)$$

where T is the number of time steps, N the number of spatial nodes (e.g., sensors or road segments), and F the number of features per

node, the objective is to forecast:

$$\mathbf{Y} = \{\mathbf{Y}_{T+1}, \dots, \mathbf{Y}_{T+H}\} \in \mathbb{R}^{H \times N \times 1}, \quad (2)$$

representing traffic states at each node for the next H steps. If a spatial graph is available, defined by an adjacency matrix $A \in \mathbb{R}^{N \times N}$, the task becomes:

$$\hat{\mathbf{Y}}_{T+1:T+H} = f_{\theta}(\mathbf{X}_1 : T, A), \quad (3)$$

where f_{θ} is a trainable model capturing both spatial and temporal patterns, acting as a parametric mapping $f_{\theta} : \mathbb{R}^{T \times N \times F} \rightarrow \mathbb{R}^{H \times N \times 1}$ that transforms historical observations into future traffic states.

Modular Modeling. To accommodate diverse traffic prediction requirements, we adopt a modular design where models are constructed from three primary components:

- **Spatial Module ($\mathcal{S}$):** Learns spatial dependencies across spatial nodes, using GCN, Diffusion GCN, or adaptive variants.
- **Temporal Module ($\mathcal{T}$):** Captures sequential traffic dynamics at each node. Examples include GRU, TCN, and Transformer.
- **Fusion Module ($\mathcal{F}$):** Integrates temporal and spatial representations via addition or concatenation.

This modularity enables flexible and reusable composition of architectures tailored to different traffic scenarios.

Code Assembly Objective. ST-LEGO aims to automate the generation of executable traffic prediction models directly from natural language descriptions. Users can provide task requirements in free-form text, without the need for structured input formats. Given:

Input: $\mathcal{P} \rightarrow$ Task Description

Output: `STModel.py` $\in$ Executable PyTorch Code

ST-LEGO translates the input into deployable model code, enabling rapid, reproducible, and adaptive development.

3 ST-LEGO Framework

To address the challenges of heterogeneous traffic prediction tasks, we introduce ST-LEGO, a principled multi-agent framework that formalizes model construction as a language-driven modular composition problem. ST-LEGO LLMs to align task semantics with architectural constraints, interprets dataset statistics as inductive signals, and produces complete, executable models with theoretical transparency and reproducibility. Inspired by the principle of assembling LEGO blocks, the framework incrementally constructs network architectures by composing reusable modules. As shown in **Figure 2**, ST-LEGO is organized around three core stages: (i) task understanding and alignment, (ii) modular assembly, and (iii) code generation. These stages are realized by three collaborative agents that jointly transform task descriptions into deployable architectures. This design not only enables flexible model construction but also highlights ST-LEGO as a new paradigm where language serves as a high-level planner and modules as verifiable building blocks.

3.1 Task Understanding and Alignment

The first stage of ST-LEGO is task understanding and alignment, which transforms free-form task descriptions and dataset metadata into structured architectural constraints. To this end, the Prompt Agent parses two complementary sources of information: semantic signals and statistical signals. Unlike prior systems constrained by

pre-formatted prompts, ST-LEGO leverages the semantic reasoning capability of LLMs to interpret natural language inputs and automatically extract core requirements, such as the presence of spatial structures (graphs or adjacency matrices), the inclusion of external factors (periodicity), efficiency requirements (e.g., real-time prediction), and user-specified model preferences. Formally, the semantic parsing process is defined as:

$$\phi_{\text{sem}} : \mathcal{P}_{\text{text}} \mapsto C_{\text{sem}}, \quad (4)$$

where $\mathcal{P}_{\text{text}}$ denotes the natural language description and C_{sem} represents the extracted set of semantic constraints.

In parallel, the framework automatically loads the target dataset and computes a set of task-level statistics that serve as inductive signals, bridging data characteristics with model design. To establish a principled connection between statistical signals and architectural choices, we explicitly distinguish four categories of coefficients:

Global Temporal Volatility (CV_{global}). Quantifies overall temporal variability:

$$CV_{\text{global}} = \frac{\text{std}(X_{1:T})}{\text{mean}(X_{1:T})}. \quad (5)$$

Larger values indicate stronger fluctuations, motivating the use of more expressive temporal encoders such as TCNs or Transformers.

Node-level variation (CV_{node}). Measures intra-node variability:

$$CV_{\text{node}} = \frac{1}{N} \sum_{i=1}^N \frac{\text{std}(X_{1:T,i})}{\text{mean}(X_{1:T,i})}. \quad (6)$$

Higher values imply stronger node-specific dynamics, indicating the need for more personalized modeling to avoid overly shared representations.

Spatial Heterogeneity (SH). Captures cross-node distributional disparity:

$$SH = \frac{\text{std}(\mu_i)}{\text{mean}(\mu_i)}, \quad \mu_i = \frac{1}{T} \sum_{t=1}^T X_{t,i}. \quad (7)$$

Larger SH values suggest the necessity of learnable adjacency or attention-based spatial encoders to adaptively capture non-uniform spatial patterns.

Average One-lag Autocorrelation Coefficient (ρ_1). Quantifies temporal persistence.

$$\rho_1 = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{t=1}^{T-1} (X_{t,i} - \bar{X}_i)(X_{t+1,i} - \bar{X}_i)}{\sum_{t=1}^T (X_{t,i} - \bar{X}_i)^2}. \quad (8)$$

Higher ρ_1 indicates stronger sequential continuity, favoring recurrent structures such as GRUs.

Sampling Interval (Δt). Extracted from timestamps to provide guidance on temporal granularity and computational requirements.

Since traffic datasets often provide only limited historical windows, these statistics may be unstable in estimation. To address this, ST-LEGO incorporates bootstrap resampling and exponential smoothing for robust estimation. Moreover, the inherent periodicity of traffic data further enhances the reliability of sample-based statistics, ensuring stable features even under short historical contexts.

Finally, the Prompt Agent fuses semantic constraints C_{sem} with statistical signals C_{stat} to form the complete constraint set:

$$C = C_{\text{sem}} \cup C_{\text{stat}}, \quad (9)$$

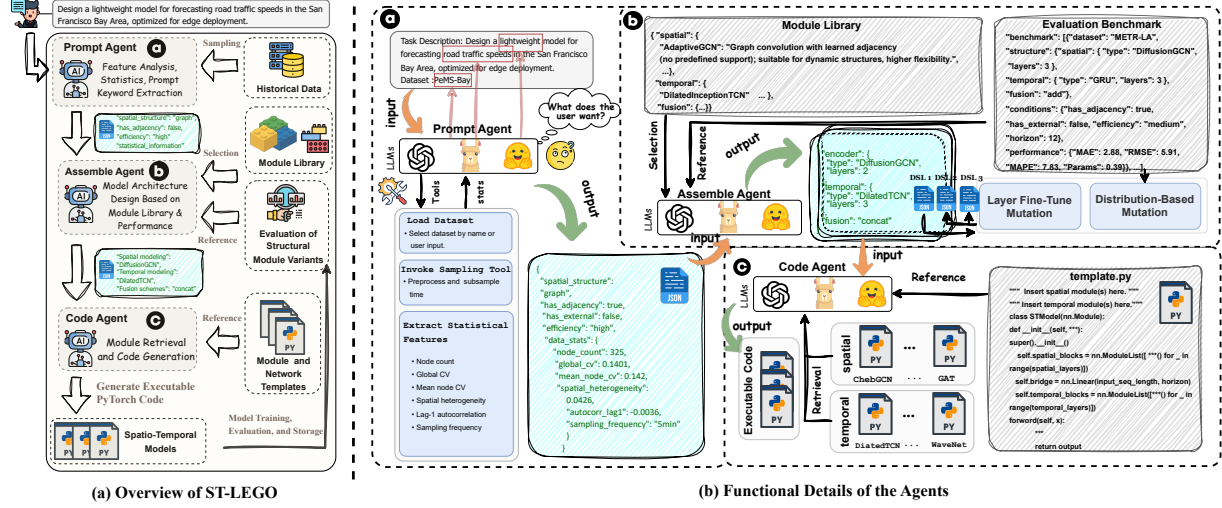


Figure 2: An overview and detailed workflow of the ST-LEGO framework. (a) presents a high-level overview of the entire pipeline from task description to model generation. (b) illustrates the detailed operational logic of each agent, including statistical analysis, module selection, structural assembly, and code generation, along with representative intermediate DSL outputs.

which is then transformed into the intermediate DSL representation, ensuring that user intent and data characteristics remain consistent and interpretable throughout subsequent modular assembly.

Table 1: Module Library and Sources.

Category	Module	Source
Spatial	GCN	GCN [19]
	GAT	GAT [45]
	AdaptiveGCN	Graph WaveNet [49]
	ChebGCN	STGCN [55]
	DiffusionGCN	D2STGNN [40]
	GumbelGraph	GTS [36]
	MixHopGCN	MixHop [1]
Temporal	SpatialAttention	STTN [50]
	SpatialEmbedding	STID [38]
	LSTM	LSTM [12]
	GRU	GRU [6]
	GLUTCN	STGCN [55]
	DilatedTCN	TCN [3]
	DilatedInceptionTCN	MTGNN [48]
	ResidualMLP	STID [38]
	TemporalAttention	STTN [50]
	Transformer	Transformer [44]
	WaveNet	Graph WaveNet [49]

3.2 Modular Assembly

The Assemble Agent serves as the core reasoning module of ST-LEGO, responsible for transforming task constraints and data characteristics into composable architectural blueprints. Rather than treating models as ad-hoc stacks of components, we view them as algebraic systems composed of standardized operators. Through formalized interfaces and evolutionary mechanisms, the Assemble Agent is able to generate predictive models that are both task-compliant and extensible.

3.2.1 Module Selection and Design. We systematically decompose mainstream traffic prediction models into three fundamental module types: spatial modules ($\mathcal{S}$), which capture dependencies across

nodes; temporal modules ($\mathcal{T}$), which model sequential dynamics at each node; and fusion modules ($\mathcal{F}$), which integrate spatio-temporal features into a unified latent representation. We survey existing architectures, extract decomposable substructures, and design standardized interface specifications for each module, thereby ensuring consistency in functional semantics, input–output formats, and contextual dependencies. As shown in **Table 1**, we curate a diverse set of spatial and temporal modules from prior literature and standardize their interfaces as reusable building blocks.

All composed models adhere to a unified input–output signature $(\mathcal{X}, \mathcal{Y})$, where $\mathcal{X} \in \mathbb{R}^{T \times N \times F}$ and $\mathcal{Y} \in \mathbb{R}^{H \times N \times 1}$. This design formalizes architecture composition as:

$$f_{\text{arch}} = \mathcal{F} \circ (\mathcal{S} \otimes \mathcal{T}), \quad (10)$$

where $\otimes$ denotes parallel combination of spatial and temporal modeling, and $\circ$ represents sequential fusion. Compared to ad-hoc assembly, this operator-based abstraction guarantees semantic and dimensional consistency across modules.

3.2.2 Bridging Spatial and Temporal Representations. To address the dimensional mismatch between spatial module outputs $\mathbf{H}_{\text{sp}} \in \mathbb{R}^{T \times N \times d_s}$ and temporal module inputs $\mathbb{R}^{T \times N \times d_t}$, we design an explicit bridging operator:

$$\tilde{\mathbf{H}}_{\text{sp}} = \mathbf{H}_{\text{sp}} \mathbf{W}_{\text{bridge}} + \mathbf{b}, \quad \mathbf{W}_{\text{bridge}} \in \mathbb{R}^{d_s \times d_t}. \quad (11)$$

This transformation semantically aligns spatial and temporal representations, ensuring composability between heterogeneous modules.

3.2.3 Frequency Reconciliation and Fusion Strategies. Spatial modeling captures low-frequency patterns, while temporal modeling focuses on high-frequency dynamics. Fusion is task-dependent, with interfaces reserved for incorporating external variables. We define two generic fusion strategies:

- (1) **Addition Fusion:** Suitable when spatial and temporal features share semantic equivalence or structural symmetry:

$$\mathbf{H}_{\text{out}} = \tilde{\mathbf{H}}_{\text{sp}} + \tilde{\mathbf{H}}_{\text{tm}} \quad (12)$$

- (2) **Concatenation Fusion:** Preserves modality-specific semantics and is appropriate for heterogeneous inputs:

$$\mathbf{H}_{\text{cat}} = [\tilde{\mathbf{H}}_{\text{sp}}; \tilde{\mathbf{H}}_{\text{tm}}] \in \mathbb{R}^{T \times N \times 2d}, \quad (13)$$

$$\mathbf{H}_{\text{out}} = \text{ReLU}(\mathbf{W}_c \cdot \mathbf{H}_{\text{cat}} + \mathbf{b}_c) \quad (14)$$

In addition, the Assemble Agent treats architectural choices such as layer depth and fusion strategy as tunable hyperparameters, recommending suitable configurations based on requirement specifications and statistical indicators. To avoid memory overflow, the maximum depth is capped at four, yet the stacking of modules—much like LEGO blocks—still enables cumulative expressiveness. This perspective frames depth and fusion not as fixed design decisions but as compositional hyperparameters, preserving compatibility with downstream prediction heads while enhancing structural flexibility and generalization.

3.2.4 Benchmark Updating Strategy. During structure generation, the Assemble Agent references a benchmark prior $\mathcal{B}$, which consists of historically strong architectural patterns across diverse datasets and tasks:

$$P(f) \propto \exp(\text{Perf}_{\mathcal{B}}(f)), \quad (15)$$

where $\text{Perf}_{\mathcal{B}}(f)$ denotes the historical performance of structure f . Unlike static search spaces, $\mathcal{B}$ adopts a dynamic updating mechanism: newly generated structures are evaluated on train-validation splits, and if they outperform existing entries, they are incorporated into the benchmark. This prevents test data leakage and ensures that the Assemble Agent’s prior knowledge remains up-to-date.

3.2.5 Mutation Strategies. To avoid premature convergence and ensure diversity, the Assemble Agent introduces two perturbation operators under benchmark guidance:

Layer Fine-Tune Mutation: This strategy is inspired by the non-linear relationship between neural network capacity and generalization ability. Referencing hierarchical perturbation approaches in neural architecture search [61], the encoder and temporal layers within the current structure are adjusted slightly:

$$L' = \min(\max(L + \Delta L, 1), L_{\max}), \quad \Delta L \in \{-1, +1\} \quad (16)$$

Here, L denotes the original number of layers and $L_{\max}$ is the maximum depth constraint. This strategy adjusts model capacity without altering the module types, making it suitable for low-cost refinement during the fine-tuning phase.

Distribution-Based Mutation: Based on the empirical usage distribution of spatial and temporal modules in a benchmark performance repository, the selection probability is defined as:

$$P(m) = \frac{f(m)}{\sum_{m' \in \mathcal{M}} f(m')} \quad (17)$$

Here, $f(m)$ denotes the frequency of module m in historical models, and $\mathcal{M}$ represents the set of available modules. This strategy guides the model structure toward empirically optimal regions via probabilistic resampling, thereby enhancing search efficiency from a structure exploration perspective.

In summary, the Assemble Agent integrates modular design, bridging and fusion mechanisms, dynamic benchmark updating, and evolutionary perturbation operators to establish a principled yet practical pipeline for structure generation. This design not only

ensures interpretability and controllability, but also enables ST-LEGO to continuously evolve toward high-performing architectures for traffic prediction.

3.3 Code Retrieval and Generation

The Code Agent is responsible for translating the intermediate DSL generated by the Assemble Agent into executable model code, serving as the final stage in the closed-loop modeling pipeline of ST-LEGO. Its primary objective is not only to automate the transition from high-level structural intent to concrete implementation, but also to guarantee semantic consistency, traceability, and formal alignment between the generated code and the structural blueprint, thereby providing verifiability for end-to-end model generation.

Unlike black-box prompt-to-code approaches [22], the Code Agent is designed as a formal compiler operating under the strict constraints of DSL. The DSL functions not merely as an intermediate structural language, but as a bridge that ensures a one-to-one correspondence between abstract architectures and executable implementations. Guided by DSL, the Code Agent completes model realization through three well-defined steps—module planning, component retrieval, and code generation—avoiding the uncontrollability and inconsistency often observed in direct code generation.

Specifically, the Code Agent consists of three stages:

- (1) **Structure Parsing:** Parses the hierarchical and dependency information in the DSL, identifying module types, parameter configurations, and inter-module connections, thereby constructing a structured implementation blueprint.
- (2) **Component Retrieval:** Based on the module type and functional descriptions, the appropriate implementation is retrieved from the module library. This includes module definitions, logic implementations, and forward computation templates, ensuring that each code segment coherently contributes to the complete model structure.
- (3) **Network Generation:** According to module order and interface specifications, automatically generates a customized STModel class. The process includes the `__init__()` constructor, `forward()` logic, configuration metadata, model naming, and alias mechanisms, ensuring that the output model carries complete execution semantics.

From a design perspective, the Code Agent emphasizes structure-driven implementation: all modules are encapsulated as classes that conform to a unified interface protocol. This design not only ensures executability but also facilitates inspection, debugging, and semantic traceability, thereby enhancing the controllability and transparency of the generation process.

This cooperative generation paradigm achieves a clear separation between structural decisions and implementation logic: the DSL provides formalized structural constraints, the template engine guarantees implementation consistency, and the LLM assists in semantic alignment and prompt interpretation. As a result, code generated under the same DSL specification is stable, consistent, and reproducible, making the Code Agent a critical final component for achieving end-to-end model generation in ST-LEGO.

4 Experiments

We evaluate ST-LEGO from three core perspectives: structural assembly, forecasting performance, and code generation reliability. Our study is guided by the following research questions:

- **(Q1)** Can ST-LEGO consistently generate valid, coherent, executable, and trainable forecasting models?
- **(Q2)** Do ST-LEGO-assembled models match or surpass the performance of manually designed models?
- **(Q3)** Does ST-LEGO exhibit strong structural diversity and adaptability across tasks?
- **(Q4)** Do structural mutation strategies (LLM-guided, layer tuning, distribution-based) improve architecture quality and forecasting accuracy?
- **(Q5)** How do key components—such as statistical analysis, mutation, and fusion strategies—impact overall performance?

We conduct comprehensive experiments covering structural verification, performance comparison, mutation analysis, component ablations, and system efficiency analysis.

4.1 Evaluation Metrics.

We benchmark ST-LEGO on six standard spatio-temporal datasets [37], covering both **traffic speed** (METR-LA, PEMS-BAY) and **traffic flow** (PEMS03, PEMS04, PEMS07, PEMS08) prediction tasks. Detailed dataset statistics are provided in **Appendix F**. The evaluation metrics fall into three main categories:

Structural assembly: Validity (DSL syntax compliance), task match (alignment with prompt constraints), executability (successful initialization and inference), and convergence (training completes within 100 epochs).

Accuracy: MAE, RMSE, and MAPE, detailed in **Appendix G**.

Structural diversity: Based on the distribution and proportion of module types across generated architectures.

4.2 Baseline Comparisons.

We compare ST-LEGO with 18 representative traffic prediction models, covering three categories: **Graph-based models:** STGCN [55], DCRNN [26], ASTGCN [14], GWNet [49], STSGCN [41], DGCRN [21], D²STGNN [40], AGCRN [2], MTGNN [48], STGODE [10], StemGNN [4], GTS [40], MegaCRN [17]. **Transformer and pre-train-based models:** STAEformer [31], STEP [39]. **Efficient modeling methods:** STNorm [7], STID [38], TimeMixer [46].

These baselines differ in architectural design and generalization capabilities. Detailed descriptions and configurations are included in the **Appendix H**.

4.3 Experimental Setup.

All experiments were conducted on a computing platform equipped with an Intel Xeon CPU Max 9462 (2.70GHz) and six NVIDIA A100 80GB SXM GPUs. We utilized four mainstream large language models (LLMs): LLaMA3-8B and Qwen2.5-7B were accessed via local deployment using the vLLM inference framework, while GPT-3.5 and GPT-4o were accessed through the OpenAI API. The source code for ST-LEGO is publicly available at our repository¹. The

spatio-temporal forecasting datasets and a subset of baseline models were obtained from publicly available benchmarks and official implementations released by the authors of related literature.

All models were trained using the Adam optimizer for up to 100 epochs with early stopping (patience = 10) to prevent overfitting. Network depth, fusion strategies, and other hyperparameters were automatically extracted from the DSL configurations generated by the Assemble Agent, ensuring that model training strictly followed the assembled structural specifications. Structure assembly experiments were initiated through natural language prompts, and both architecture generation and code synthesis were executed automatically within ST-LEGO's multi-agent collaborative framework.

4.4 Assembly Quality Verification (Q1).

We evaluate ST-LEGO's structural robustness under two types of prompts: efficiency-oriented and capacity-oriented, each with multiple variants. For each LLM backbone (LLaMA3.1-8B, Qwen2.5-7B, GPT-3.5, GPT-4o), we generate 54 models, yielding 216 in total, and assess structural validity, task alignment, code executability, and convergence.

As shown in **Figure 3**, ST-LEGO achieves 100% structural validity and code executability across all cases. This robustness stems from the carefully standardized module interfaces, which guarantee tensor shape consistency during computation and transmission—an assurance that black-box code generation approaches cannot provide. Under efficiency-oriented prompts, most models converge successfully while satisfying the parameter constraint ($<0.1M$), with GPT-3.5 and GPT-4o reaching convergence rates of 100% and 98.15%, respectively. Capacity-oriented prompts present greater difficulty: LLaMA3.1-8B satisfies task constraints in only 62.96% of cases, whereas GPT-3.5 and GPT-4o again achieve 100%. These findings demonstrate both the reliability of ST-LEGO's assembly pipeline and the superior structural reasoning ability of GPT-based agents, particularly under complex capacity-oriented tasks.

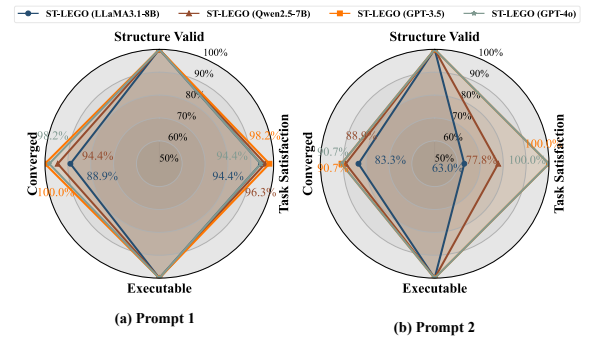


Figure 3: Assembly quality under different prompt types.

4.5 Forecasting Accuracy Comparison (Q2).

We compare ST-LEGO with 18 baseline models across six benchmark datasets covering both traffic flow and traffic speed prediction. As shown in **Table 2**, ST-LEGO consistently achieves strong forecasting performance without manual tuning, obtaining the best MAE or MAPE on all datasets. Moreover, most of the second-best results also originate from ST-LEGO-generated models. This advantage can be attributed to the benchmark priors, which provide references to historically effective module combinations, together

¹<https://github.com/ShuhaoLi/ST-LEGO>

Table 2: Forecasting accuracy comparison between ST-LEGO-generated models and manually designed baselines. Best results are in bold, and second-best are *in italics*.

Datasets	METR-LA			PeMS-BAY			PEMS03			PEMS04			PEMS07			PEMS08		
Metrics	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
STGCN	3.21	6.37	8.56%	1.67	3.76	3.78%	15.81	27.55	16.20%	19.93	31.68	13.49%	22.29	35.92	9.65%	16.3	25.5	10.80%
DCRNN	3.02	6.25	8.50%	1.61	3.67	3.72%	15.55	27.37	15.82%	19.67	31.32	13.58%	21.11	34.12	9.10%	15.36	24.18	10.17%
ASTGCN	3.82	7.53	10.42%	1.96	4.37	4.33%	16.16	28.12	17.85%	20.40	32.47	15.11%	23.75	36.58	11.84%	15.90	23.58	11.48%
GWNet	3.02	6.22	8.32%	1.68	3.85	3.67%	14.65	25.43	15.25%	18.71	30.25	13.90%	20.62	33.34	9.35%	14.86	23.64	10.12%
STSGCN	3.61	7.20	9.87%	1.69	3.94	3.87%	15.39	26.95	15.84%	19.23	31.67	13.79%	21.57	35.21	9.12%	14.64	22.44	9.62%
DGCRN	2.9	6.02	7.82%	1.57	3.78	3.60%	14.75	26.21	14.72%	18.99	30.45	12.95%	20.1	33.3	9.31%	14.79	23.94	10.38%
D ² STGNN	3.05	5.93	7.80%	1.67	3.56	3.50%	15.02	26.38	15.36%	18.41	29.88	12.65%	19.68	32.49	9.01%	14.6	23.48	10.30%
AGCRN	3.26	6.48	9.00%	1.57	3.64	3.61%	15.19	26.68	14.56%	19.36	30.29	13.31%	20.87	34.53	9.68%	15.25	24.78	10.49%
MTGNN	3.18	6.22	8.22%	1.8	3.74	3.57%	14.93	25.25	15.42%	19.14	29.93	13.38%	21.17	34.31	9.39%	15.41	24.41	10.23%
STGODE	3.05	6.41	8.94%	1.61	3.84	3.68%	15.46	26.19	16.58%	18.91	30.86	13.66%	20.44	33.87	9.09%	14.36	21.75	9.32%
StemGNN	3.68	7.48	10.41%	2.13	4.52	4.57%	15.46	25.1	15.36%	22.95	32.36	16.67%	22.52	36.25	10.43%	14.88	26.43	12.97%
GTS	3.32	6.32	8.69%	1.68	3.92	3.89%	14.88	25.42	15.01%	21.49	31.33	14.94%	22.52	35.58	9.70%	15.07	26.77	10.95%
STEP	3.09	6.04	7.96%	1.43	3.53	3.34%	15.3	26.01	14.48%	18.5	30.03	12.58%	20.55	32.54	8.75%	15.35	23.51	9.58%
STNorm	3.14	6.59	9.02%	1.73	3.82	3.60%	15.35	27.36	16.48%	19.32	32.25	13.09%	20.62	34.9	9.28%	14.4	24.85	9.38%
STID	3.12	6.45	9.14%	1.69	3.66	3.71%	15.52	27.42	16.37%	18.38	29.85	12.52%	19.6	33.76	9.25%	14.37	23.36	9.37%
STAEformer	2.98	6.20	8.22%	1.68	3.89	3.66%	15.84	27.72	16.17%	19.30	30.67	13.32%	20.76	31.27	8.86%	14.72	21.81	9.39%
MegaCRN	2.97	6.26	8.15%	1.65	3.82	3.59%	15.07	26.70	15.27%	19.01	30.12	13.07%	19.74	30.51	8.36%	14.49	21.42	9.21%
TimeMixer	2.84	5.63	8.18%	1.62	3.80	3.57%	14.40	24.28	15.32%	18.47	30.81	13.17%	19.03	31.53	8.01%	14.08	21.99	9.19%
ST-LEGO (LLaMA3.1-8B)	2.59	5.38	7.22%	1.81	4.38	4.03%	13.62	24.11	14.33%	18.61	29.81	12.67%	18.49	30.61	7.61%	13.35	20.22	8.70%
ST-LEGO (Qwen2.5-7B)	2.93	5.88	8.17%	1.35	3.17	3.02%	10.63	21.80	11.18%	18.70	29.18	12.73%	18.26	30.26	7.52%	11.03	18.93	7.18%
ST-LEGO (GPT-3.5)	2.85	5.94	7.94%	1.63	3.69	3.63%	14.57	25.51	15.33%	18.28	29.61	12.44%	19.71	32.82	8.11%	10.97	18.81	7.15%
ST-LEGO (GPT-4o)	2.87	5.94	8.00%	1.59	3.68	3.57%	14.28	24.25	15.03%	16.98	27.82	11.57%	17.64	29.90	7.26%	10.16	16.51	6.62%

with mutation strategies that ensure the Assemble Agent maintains or improves predictive performance across generations.

Specifically, LLaMA3.1-8B leads on METR-LA with 2.59 MAE and 5.38 RMSE. Qwen2.5-7B excels on PeMS-BAY and PEMS03, showing robustness under noisy conditions. GPT-4o demonstrates consistent strength across PEMS04 to PEMS08, achieving 10.16 MAE and 6.62% MAPE on PEMS08. GPT-4o and GPT-3.5 yield the most stable and generalizable structures. Qwen2.5-7B is particularly effective for low-frequency tasks with compact models. LLaMA3.1-8B performs well on specific datasets but shows higher variability. These results validate ST-LEGO’s adaptability, demonstrating its ability to achieve high accuracy across heterogeneous datasets and diverse forecasting scenarios.

4.6 Analysis of Structural Diversity and Task Adaptability (Q3).

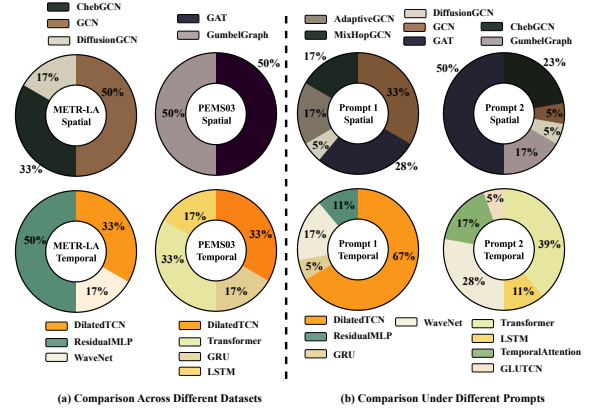
We analyze ST-LEGO’s structural choices using GPT-4o across two datasets (METR-LA, PEMS03) and multiple prompts. As shown in **Figure 4**, different datasets lead to distinct module selections: ChebGCN and DiffusionGCN are preferred in METR-LA, while GAT and GumbelGraph dominate in PEMS03. WaveNet appears in both, suggesting shared temporal structure preference.

Prompt semantics also influence architecture: efficiency-oriented prompts favor GRU and DilatedTCN, while expressiveness-oriented prompts guide selection toward Transformer and GLU-TCN. These results confirm ST-LEGO’s ability to adaptively compose diverse structures aligned with both data and task requirements.

4.7 Structural Mutation Strategies Study (Q4).

We evaluate three mutation strategies—LLM direct generation, layer fine-tuning, and distribution-based mutation—by tracking their contributions to top-performing models across datasets and LLMs, as shown in **Figure 5**.

LLM direct generation dominates for GPT-3.5 and GPT-4o, while mutation strategies provide greater value for Qwen and LLaMA. Distribution-based mutation improves robustness on PeMS03 and

**Figure 4: Comparison of structural diversity and task adaptability under different datasets and prompt conditions.**

PeMS08, and layer tuning proves helpful in capacity-sensitive tasks like METR-LA. These strategies enhance the search space and support adaptive model refinement.

4.8 Module Importance Ablation Study (Q5).

We ablate three key components: statistical analysis, mutation strategies, and fusion mechanisms. Results on METR-LA (**Table 3**) show that removing statistical features leads to poorer structure-task alignment (e.g., GPT-4o MAE $\uparrow$ 0.69). Omitting mutation causes significant drops for GPT-3.5 (MAE $\uparrow$ to 2.92). Fusion removal leads to the worst degradation (e.g., GPT-3.5 MAE = 3.63), confirming the necessity of spatio-temporal integration. These findings highlight the importance of each module: statistics and fusion directly boost accuracy, while mutation improves structural generalization. Together, they validate the effectiveness of ST-LEGO’s language-driven modular paradigm. Additional ablation results on PeMS03 are provided in **Appendix A**.

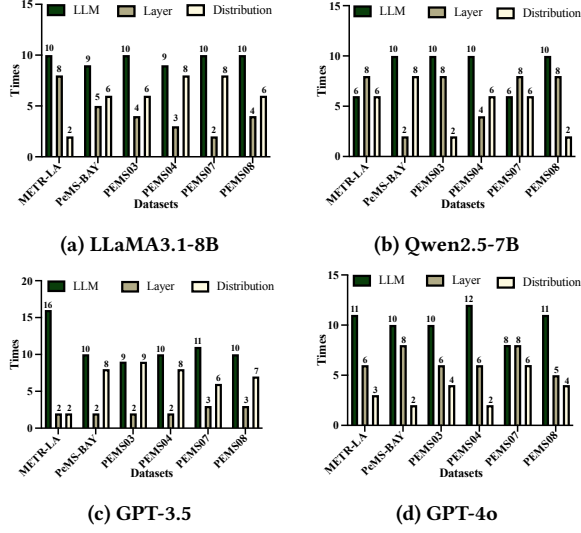


Figure 5: Distribution of best-performing results across different LLM backbones.

Table 3: Ablation study.

Variants	w/o Statistical Information			w/o Mutation		
Foundation LLMs	MAE	RMSE	MAPE	MAE	RMSE	MAPE
ST-LEGO (LLaMA3.1-8B)	3.15	6.45	8.67%	2.59	5.38	7.22%
ST-LEGO (Qwen2.5-7B)	3.52	7.24	9.81%	2.93	5.88	8.18%
ST-LEGO (GPT-3.5)	3.58	7.12	9.53%	2.92	5.88	8.14%
ST-LEGO (GPT-4o)	3.56	7.13	9.81%	2.87	5.94	8.00%

Variants	w/o Fusion			w/o None		
Foundation LLMs	MAE	RMSE	MAPE	MAE	RMSE	MAPE
ST-LEGO (LLaMA3.1-8B)	2.83	5.97	8.02%	2.59	5.38	7.22%
ST-LEGO (Qwen2.5-7B)	3.25	6.51	9.07%	2.93	5.88	8.17%
ST-LEGO (GPT-3.5)	3.63	6.59	8.76%	2.85	5.94	7.94%
ST-LEGO (GPT-4o)	3.18	6.59	8.81%	2.87	5.94	8.00%

5 Related Works

5.1 Traffic Prediction

Traffic prediction is essential for web-based intelligent transportation services, including online navigation platforms, real-time traffic management, and smart city applications, requiring joint modeling of temporal dynamics and spatial dependencies. Early models such as STGCN and DCRNN combined graph convolution and recurrent networks [55]. Later methods like AGCRN, GWNet, and MTGNN introduced modular architectures with node adaptivity, multi-view fusion, and graph generation [2, 48]. Recent work explores more efficient paradigms: STNorm and STID adopt MLP-based designs [7, 38], while Transformer-based STAEformer and STEP improve long-range modeling and generalization through attention and pre-training [31, 39]. With the emergence of large language models, recent studies apply LLMs to traffic prediction. ST-LLM and ST-LLM⁺ leverage LoRA or freezing to encode grid-based sequences [28, 29]; STG-LLM and IMPEL generate region-level text or knowledge graph embeddings to enhance robustness under cold-start or transfer settings [33, 35]; UrbanGPT and xTP-LLM reformulate prediction as a language generation task, targeting zero-shot and explainability [15, 27]. However, these approaches lack structural control, executable code, and reusable modules, and do not offer a unified pipeline for model construction. ST-LEGO addresses these

gaps by proposing an LLM-based framework for modular composition and automatic model generation.

5.2 LLMs for NAS and Code Generation.

Large language models (LLMs) have demonstrated strong generalization across natural language processing, program synthesis, and multi-modal reasoning tasks. Recent work extends their use to structured domains, such as TabLLM for table reasoning [16], GraphQA for graph-structured queries [43], and task planning. In neural architecture search (NAS), GPT-NAS employs LLMs to explore design spaces, improving interpretability and flexibility [56]. However, these methods typically emphasize search-space encoding rather than supporting full model generation or domain-specific execution. In parallel, studies such as Code-as-Policies and CodeRL apply LLMs to generate logic-based structures or executable code [20], but they primarily address general-purpose tasks and lack integration with domain-specific modules or structured semantics. This limitation constrains their applicability to traffic prediction and other spatio-temporal settings. Overall, existing approaches often rely on fixed templates or black-box search, without leveraging LLMs as structured model constructors. ST-LEGO advances this line of work by embedding LLMs within a multi-agent framework that supports semantic interpretation, module selection, and executable code generation. By combining statistical guidance with mutation strategies, it delivers an end-to-end pipeline from language input to deployable code, extending the role of LLMs from general reasoning to domain-specific model assembly.

6 Conclusion and Future Works

In this paper, we present ST-LEGO, a modular framework powered by large language models that reformulates traffic prediction as a language-driven structural composition task. Through a multi-agent system—comprising the Prompt Agent for task parsing, the Assemble Agent for modular construction, and the Code Agent for executable code generation—ST-LEGO enables end-to-end model generation from natural language input. By decomposing models into standardized spatial, temporal, and fusion components, ST-LEGO supports flexible composition, reuse, and generalization. Experiments on real-world datasets show that ST-LEGO achieves strong accuracy, structural diversity, and convergence, with some models even surpassing state-of-the-art baselines without manual tuning. This work establishes a new paradigm for structure-aware traffic modeling and highlights the potential of LLMs in automated model generation. Future work will extend the module library to support grid-based and image-based inputs and incorporate multi-modal signals such as maps and sensor metadata to enhance task understanding. Structural partitioning strategies could also be extended to better handle ultra-large-scale datasets.

Acknowledgments

This work was supported by the China Scholarship Council (Grant No. 202506100105), the Australian Research Council (Grant Nos. DP240101322 and DP260100709), and the Shanghai Science and Technology Program (Grant No. 22dz1203200). The computations in this research were performed using the CFFF platform of Fudan University. We are also grateful for the rich set of components provided by previous studies, which facilitated this research.

References

- [1] Sami Abu-El-Hajja, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, and Aram Galstyan. 2019. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *international conference on machine learning*. PMLR, 21–29.
- [2] Lei Bai, Lina Yao, Can Li, Xianzhi Wang, and Can Wang. 2020. Adaptive graph convolutional recurrent network for traffic forecasting. *Advances in neural information processing systems* 33 (2020), 17804–17815.
- [3] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271* (2018).
- [4] Defu Cao, Yujing Wang, Juanyong Duan, Ce Zhang, Xia Zhu, Congrui Huang, Yunhai Tong, Bixiong Xu, Jing Bai, Jie Tong, et al. 2020. Spectral temporal graph neural network for multivariate time-series forecasting. *Advances in neural information processing systems* 33 (2020), 17766–17778.
- [5] Kevin Chen, Zifan Xu, Lunjun Yu, et al. 2022. Code as Policies: Language Model Programs for Embodied Control. In *ICLR*.
- [6] Kyunghyun Cho, B van Merriënboer, Caglar Gulcehre, F Bougares, H Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP 2014)*.
- [7] Jinliang Deng, Xiusi Chen, Renhe Jiang, Xuan Song, and Ivor W Tsang. 2021. St-norm: Spatial and temporal normalization for multi-variate time series forecasting. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 269–278.
- [8] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural Architecture Search: A Survey. *JMLR* 20, 55 (2019), 1–21.
- [9] Yuchen Fang, Hao Miao, Yuxuan Liang, Liwei Deng, Yue Cui, Ximu Zeng, Yuyang Xia, Yan Zhao, Torben Bach Pedersen, Christian S Jensen, et al. 2025. Unraveling Spatio-Temporal Foundation Models via the Pipeline Lens: A Comprehensive Review. *arXiv preprint arXiv:2506.01364* (2025).
- [10] Zheng Fang, Qingqing Long, Guojie Song, and Kunqing Xie. 2021. Spatial-temporal graph ode networks for traffic flow forecasting. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 364–373.
- [11] Haotian Gao, Renhe Jiang, Zheng Dong, Jinliang Deng, Yuxin Ma, and Xuan Song. 2024. Spatial-Temporal-Decoupled Masked Pre-training for Spatiotemporal Forecasting. In *IJCAI*.
- [12] Alex Graves and Alex Graves. 2012. Long short-term memory. *Supervised sequence labelling with recurrent neural networks* (2012), 37–45.
- [13] Canyang Guo, Chi-Hua Chen, Feng-Jang Hwang, Ching-Chun Chang, and Chin-Chen Chang. 2024. Multi-view spatiotemporal learning for traffic forecasting. *Information Sciences* 657 (2024), 119868.
- [14] Shengnan Guo, Youfang Lin, Ning Feng, Chao Song, and Huaiyu Wan. 2019. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 922–929.
- [15] Xusen Guo, Qiming Zhang, Junyue Jiang, Mingxing Peng, Meixin Zhu, and Hao Frank Yang. 2024. Towards explainable traffic flow prediction with large language models. *Communications in Transportation Research* 4 (2024), 100150.
- [16] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. Tabllm: Few-shot classification of tabular data with large language models. In *International conference on artificial intelligence and statistics*. PMLR, 5549–5581.
- [17] Renhe Jiang, Zhaonan Wang, Jiawei Yong, Puneet Jeph, Quanjun Chen, Yasumasa Kobayashi, Xuan Song, Shintaro Fukushima, and Toyotaro Suzumura. 2023. Spatio-temporal meta-graph learning for traffic forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 8078–8086.
- [18] Zheng Jiang, Yuhui Zhang, Zhiting Yu, et al. 2023. Gorilla: Large Language Model Connected with Massive APIs. In *ICML*.
- [19] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [20] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 21314–21328.
- [21] Fuxian Li, Jie Feng, Huan Yan, Guangyin Jin, Fan Yang, Funing Sun, Depeng Jin, and Yong Li. 2023. Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution. *ACM Transactions on Knowledge Discovery from Data* 17, 1 (2023), 1–21.
- [22] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2024. Acecoder: An effective prompting technique specialized in code generation. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–26.
- [23] Shuhao Li, Yue Cui, Libin Li, Weidong Yang, Fan Zhang, and Xiaofang Zhou. 2024. ST-ABC: Spatio-temporal attention-based convolutional network for multi-scale lane-level traffic prediction. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1185–1198.
- [24] Shuhao Li, Yue Cui, Jingyi Xu, Libin Li, Linghai Meng, Weidong Yang, Fan Zhang, and Xiaofang Zhou. 2025. Unifying lane-level traffic prediction from a graph structural perspective: Benchmark and baseline. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [25] Shuhao Li, Yue Cui, Jingyi Xu, Jing Zhao, Fan Zhang, Weidong Yang, and Xiaofang Zhou. 2024. Seeing the Forest for the Trees: Road-Level Insights Assisted Lane-Level Traffic Prediction. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1266–1275.
- [26] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations*.
- [27] Zhonghang Li, Lianghao Xia, Jiabin Tang, Yong Xu, Lei Shi, Long Xia, Dawei Yin, and Chao Huang. 2024. Urbangpt: Spatio-temporal large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5351–5362.
- [28] Chenxi Liu, Kethmi Hirushini Hettige, Qianxiong Xu, Cheng Long, Shili Xiang, Gao Cong, Ziyue Li, and Rui Zhao. 2025. ST-LLM+: Graph Enhanced Spatio-Temporal Large Language Models for Traffic Prediction. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [29] Chenxi Liu, Sun Yang, Qianxiong Xu, Zhishuai Li, Cheng Long, Ziyue Li, and Rui Zhao. 2024. Spatial-temporal large language model for traffic prediction. In *2024 25th IEEE International Conference on Mobile Data Management (MDM)*. IEEE, 31–40.
- [30] Changan Liu, Ahad N Zehmakan, and Zhongzhi Zhang. 2024. Fast Query of Biharmonic Distance in Networks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1887–1897.
- [31] Hangchen Liu, Zheng Dong, Renhe Jiang, Jiewen Deng, Jinliang Deng, Quanjun Chen, and Xuan Song. 2023. Spatio-temporal adaptive embedding makes vanilla transformer sota for traffic forecasting. In *Proceedings of the 32nd ACM international conference on information and knowledge management*. 4125–4129.
- [32] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2019. DARTS: Differentiable Architecture Search. In *ICLR*.
- [33] Lei Liu, Shuo Yu, Runze Wang, Zhenxun Ma, and Yanming Shen. 2024. How can large language models understand spatial-temporal data? *arXiv preprint arXiv:2401.14192* (2024).
- [34] Xunlian Luo, Chunjiang Zhu, Detian Zhang, and Qing Li. 2023. STG4Traffic: A Survey and Benchmark of Spatial-Temporal Graph Neural Networks for Traffic Prediction. *arXiv preprint arXiv:2307.00495* (2023).
- [35] Tong Nie, Junlin He, Yuewen Mei, Guoyang Qin, Guilong Li, Jian Sun, and Wei Ma. 2025. Joint estimation and prediction of city-wide delivery demand: A large language model empowered graph-based learning approach. *Transportation Research Part E: Logistics and Transportation Review* 197 (2025), 104075.
- [36] Chao Shang, Jie Chen, and Jinbo Bi. 2021. Discrete Graph Structure Learning for Forecasting Multiple Time Series. In *International Conference on Learning Representations*.
- [37] Zezhi Shao, Fei Wang, Yongjun Xu, Wei Wei, Chengqing Yu, Zhao Zhang, Di Yao, Tao Sun, Guangyin Jin, Xin Cao, et al. 2025. Exploring Progress in Multivariate Time Series Forecasting: Comprehensive Benchmarking and Heterogeneity Analysis. *IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING* 37, 1 (2025), 291.
- [38] Zezhi Shao, Zhao Zhang, Fei Wang, Wei Wei, and Yongjun Xu. 2022. Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 4454–4458.
- [39] Zezhi Shao, Zhao Zhang, Fei Wang, and Yongjun Xu. 2022. Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 1567–1577.
- [40] Zezhi Shao, Zhao Zhang, Wei Wei, Fei Wang, Yongjun Xu, Xin Cao, and Christian S Jensen. 2022. Decoupled dynamic spatial-temporal graph neural network for traffic forecasting. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2733–2746.
- [41] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. 2020. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 914–921.
- [42] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2024. Table meets llm: Can large language models understand structured table data? a benchmark and empirical study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 645–654.
- [43] Dhaval Taunk, Lakshya Khanna, Siri Venkata Pavan Kumar Kandru, Vasudeva Varma, Charu Sharma, and Makarand Tapaswi. 2023. Grapeqa: Graph augmentation and pruning to enhance question-answering. In *Companion Proceedings of the ACM Web Conference 2023*. 1138–1144.
- [44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [45] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).

- [46] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. *arXiv preprint arXiv:2405.14616* (2024).
- [47] Michael Wooldridge. 2009. *An Introduction to MultiAgent Systems*. John Wiley & Sons.
- [48] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. 2020. Connecting the dots: Multivariate time series forecasting with graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 753–763.
- [49] Z Wu, S Pan, G Long, J Jiang, and C Zhang. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. In *The 28th International Joint Conference on Artificial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence Organization.
- [50] Mingxing Xu, Wenrui Dai, Chunmiao Liu, Xing Gao, Weiyao Lin, Guo-Jun Qi, and Hongkai Xiong. 2020. Spatial-temporal transformer networks for traffic flow forecasting. *arXiv preprint arXiv:2001.02908* (2020).
- [51] Yuanyuan Xu, Hanchen Wang, Wenjie Zhang, Lexing Xie, Yin Chen, Flora Salim, Ying Zhang, Justin Gooding, and Toby Walsh. 2025. AI-Empowered Catalytic Discovery: A Survey from Classical Machine Learning Approaches to Large Language Models. *arXiv preprint arXiv:2502.13626* (2025).
- [52] Yuanyuan Xu, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2025. UniDyG: A Unified and Effective Representation Learning Approach for Large Dynamic Graphs. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [53] Yuanyuan Xu, Wenjie Zhang, Ying Zhang, Xiwei Xu, and Xuemin Lin. 2025. Fast and accurate temporal hypergraph representation for hyperege prediction. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 1727–1738.
- [54] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Shaochen Zhong, Bing Yin, and Xia Hu. 2024. Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data* 18, 6 (2024), 1–32.
- [55] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*. 3634–3640.
- [56] Caiyang Yu, Xianggen Liu, Yifan Wang, Yun Liu, Wentao Feng, Xiong Deng, Chenwei Tang, and Jiancheng Lv. 2024. GPT-NAS: Neural architecture search meets generative pre-trained transformer model. *Big Data Mining and Analytics* (2024).
- [57] Yuan Yuan, Jingtao Ding, Jie Feng, Depeng Jin, and Yong Li. 2024. Unist: A prompt-empowered universal model for urban spatio-temporal prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4095–4106.
- [58] Weijia Zhang, Le Zhang, Jindong Han, Hao Liu, Yanjie Fu, Jingbo Zhou, Yu Mei, and Hui Xiong. 2024. Irregular traffic time series forecasting based on asynchronous spatio-temporal graph convolutional networks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4302–4313.
- [59] Yu Zhang, Hua Lu, Ning Liu, Yonghui Xu, Qingzhong Li, and Lizhen Cui. 2024. Personalized federated learning for cross-city traffic prediction. In *33rd International Joint Conference on Artificial Intelligence, IJCAI*. 5526–5534.
- [60] Xi Zhu, Haochen Xue, Ziwei Zhao, Wujia Xu, Jingyuan Huang, Minghao Guo, Qifan Wang, Kaixiong Zhou, and Yongfeng Zhang. 2025. Llm as gnn: Graph vocabulary learning for text-attributed graph foundation models. *arXiv preprint arXiv:2503.03313* (2025).
- [61] Barret Zoph and Quoc V Le. 2017. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

A Supplementary Ablation Study

To evaluate the importance of ST-LEGO’s core components under different data conditions, we conduct ablation experiments on the PEMS03 dataset. We examine the effects of removing statistical feature extraction from the Prompt Agent (w/o Statistical Information), the structural mutation strategy (w/o Mutation), and the fusion mechanism (w/o Fusion).

As shown in **Table 4**, removing statistical guidance (w/o Statistical Information) causes a sharper performance drop on PEMS03 than on METR-LA. For example, GPT-4o’s MAE rises from 14.28 to 17.14 and MAPE by nearly 3%, whereas on METR-LA the increase was only about 1.3%. This indicates that PEMS03 has stronger node heterogeneity and temporal variability, making statistical indicators (e.g., coefficient of variation, autocorrelation) more critical

for structural adaptation. For the mutation strategy (w/o Mutation), performance degrades notably on both datasets, especially for Qwen2.5-7B on PEMS03, where MAE increases from 10.63 to 17.14, compared to only 2.93 on METR-LA. This suggests a more complex structural space in PEMS03, where mutation helps avoid suboptimal solutions. Removing the fusion mechanism (w/o Fusion) yields more consistent effects across datasets, generally producing a 1.0–2.5 increase in MAE, confirming the steady benefit of spatio-temporal fusion, particularly under asymmetric feature distributions such as in PEMS03. Compared to METR-LA, PEMS03 relies more heavily on statistical guidance and mutation strategies, highlighting ST-LEGO’s ability to adapt to heterogeneous and complex forecasting tasks.

Table 4: Abalation study on PEMS03 dataset.

Variants	w/o Statistical Information			w/o Mutation		
Foundation LLMs	MAE	RMSE	MAPE	MAE	RMSE	MAPE
ST-LEGO (LLaMA3.1-8B)	16.29	28.93	17.19%	13.62	24.11	14.33%
ST-LEGO (Qwen2.5-7B)	12.76	26.14	13.42%	17.14	25.16	18.04%
ST-LEGO (GPT3.5)	17.42	30.63	18.42%	15.07	26.16	15.87%
ST-LEGO (GPT4o)	17.14	29.10	18.04%	14.28	24.25	15.03%
Variants	w/o Fusion			w/o None		
Foundation LLMs	MAE	RMSE	MAPE	MAE	RMSE	MAPE
ST-LEGO (LLaMA3.1-8B)	15.11	26.73	15.92%	13.62	24.11	14.33%
ST-LEGO (Qwen2.5-7B)	11.79	24.21	12.42%	10.63	21.80	11.18%
ST-LEGO (GPT-3.5)	16.17	28.32	17.04%	14.57	25.51	15.33%
ST-LEGO (GPT-4o)	15.88	26.95	16.69%	14.28	24.25	15.03%

A.1 System Efficiency Analysis

We evaluate the resource consumption and response performance of ST-LEGO by measuring peak memory, inference time, and API cost across its three core agents—Prompt, Assemble, and Code—under different LLM backends.

Table 5 shows that locally deployed models generally offer lower cost and fast response in the Prompt and Assemble stages, with average latency below 1.5s. LLaMA3.1 consumed about 71,800 MB memory, slightly higher than Qwen2.5, but both delivered stable performance. For the Code Agent, which requires parsing complex structures and generating code, latency was higher: LLaMA3.1 took 34.94s and Qwen2.5 53.69s. API-based models were far faster at this stage, with GPT-3.5 averaging 2.82s and GPT-4o 4.31s, though their Prompt/Assemble latency was not significantly lower. API costs were low (0.002–0.02 per interaction), with GPT-4o achieving the best performance but also the highest cost, while Qwen2.5 offered the best trade-off locally.

Table 5: Inference cost analysis of ST-LEGO under different base LLMs.

Base LLMs	LLaMA3.1-8B		Qwen2.5-7B	
	Costs	Mem. (Peak)	Mem. (Peak)	Infer. Time
Prompt Agent		71840.42	1.25	68320.37
Assemble Agent		71865.9	0.70	68370.56
Code Agent		71536.83	34.94	68018.85
Base LLMs	GPT-3.5		GPT-4o	
	Costs	API Cost	API Cost	Infer. Time
Prompt Agent		0.002	1.21	0.010
Assemble Agent		0.002	1.14	0.010
Code Agent		0.007	2.82	0.020

B Module Library Composition

To support flexible and generalizable structure assembly, ST-LEGO constructs a standardized module library, covering mainstream spatio-temporal modeling paradigms. Each module is defined with unified interfaces and descriptive documentation, facilitating semantic parsing and combination by the Prompt and Assemble Agents. The modules are categorized into **spatial**, **temporal**, and **fusion** types, as detailed below.

Partial Module Library (JSON Format)

"spatial": "AdaptiveGCN": "Graph convolution with learned adjacency (no predefined support); suitable for dynamic structures, higher flexibility.", "ChebGCN": "Chebyshev polynomial-based graph convolution; efficient and scalable with fixed adjacency matrix.", "DiffusionGCN": "Diffusion process-based convolution with direction-aware propagation.", "MixHopGCN": "Multi-hop feature fusion graph convolution supporting hierarchical dependencies.", "GumbelGraph": "Structure learner based on Gumbel-softmax sampling.",
 "temporal": "GRU": "Gated Recurrent Unit; compact and faster than LSTM, limited in very long-term dependencies.", "DilatedTCN": "Temporal convolution with exponentially increasing dilation; fast and parallel, good for long dependencies.", "GLU-TCN": "Gated Linear Unit based Temporal Convolution.", "LSTM": "Long Short-Term Memory; handles long sequences better, but slower and more resource-intensive.", "Transformer": "Stacked attention-based encoder; powerful for long-range dependencies, more computation-heavy",
 "fusion": "add": "Element-wise addition of spatial and temporal features; fast and parameter-free.", "concat": "Concatenates features across dimensions, followed by projection; more expressive but adds parameters."

C Prompt Examples

To facilitate task-aware structure generation, ST-LEGO supports natural language prompts that flexibly express modeling intentions. The Prompt Agent does not require rigid prompt formatting and can interpret user intent through semantic parsing. Here we provide representative examples for the two main prompt categories:

C.1 Prompt Category 1: Efficiency-Oriented (Lightweight Modeling)

Example Prompt 1 (Lightweight Model)

Design a spatio-temporal forecasting model using the `{dataset_name}` dataset. The model should prioritize efficiency and parameter compactness, suitable for real-time deployment in edge environments.

This prompt instructs the system to favor lightweight components (e.g., GRU, GCN, shallow depth) and produce models with reduced parameter counts, adhering to hard resource constraints.

C.2 Prompt Category 2: Accuracy-Oriented (High Capacity Modeling)

Example Prompt 2 (High-Accuracy Model)

Design a high-accuracy forecasting model for traffic speed/flow prediction on the `{dataset_name}` dataset. Please include expressive spatial-temporal modules that maximize prediction performance, even at the cost of training efficiency.

This prompt emphasizes predictive performance, encouraging the use of deeper temporal modules (e.g., Transformer, DilatedTCN) and more expressive spatial encoders (e.g., GAT, ChebGCN), aligned with benchmark-oriented objectives.

Note: The complete set of prompt templates and the performance-guided component frequency table (Benchmark JSON) used by the Assemble Agent are available in the project repository.

D ST-LEGO Main Pipeline Pseudocode

To illustrate the complete workflow of ST-LEGO, we provide the following pseudocode, which highlights the sequential and collaborative roles of Prompt Agent, Assemble Agent, and Code Agent. The pipeline formalizes language-driven model construction as a multi-stage compilation process, transforming free-form natural language descriptions into executable PyTorch code.

Algorithm 1 ST-LEGO Pipeline

Require: Natural language prompt $\mathcal{P}$, dataset $\mathcal{D}$

Ensure: Executable PyTorch code C

- 1: **Stage 1: Prompt Agent (Task Understanding)**
- 2: $C_{\text{sem}} \leftarrow \phi_{\text{sem}}(\mathcal{P})$ {semantic constraints from text: efficiency/accuracy, graph usage, etc.}
- 3: $C_{\text{stat}} \leftarrow \phi_{\text{stat}}(\mathcal{D})$ {statistics: $\text{CV}_{\text{global}}$, CV_{node} , SH , ρ_1 , Δt }
- 4: $\mathcal{T}_{\text{spec}} \leftarrow C_{\text{sem}} \cup C_{\text{stat}}$ {unified task specification}
- 5: **Stage 2: Assemble Agent (Modular Composition)**
- 6: $(\mathcal{S}, \mathcal{T}, \mathcal{F}) \sim \mathcal{A}(\mathcal{T}_{\text{spec}}, \mathcal{B})$ {module set sampled under benchmark prior $\mathcal{B}$ }
- 7: $L' = \min\{\max\{1, L + \Delta L\}, L_{\text{max}}\}$, $\Delta L \in \{-1, +1\}$ {layer fine-tuning for capacity adjustment}
- 8: $P(m) = \frac{f(m)}{\sum_{m' \in \mathcal{M}} f(m')}$, $m \sim P(\cdot)$ {distribution-guided mutation over module pool $\mathcal{M}$ }
- 9: $\tilde{\mathbf{H}}_{\text{sp}} = \mathbf{H}_{\text{sp}} \mathbf{W}_{\text{bridge}} + \mathbf{b}$, $\mathbf{W}_{\text{bridge}} \in \mathbb{R}^{d_s \times d_t}$ {shape alignment (bridge)}
- 10: $\mathcal{S}_{\text{DSL}} := (\mathcal{S}_{\text{type}}, L_S, \mathcal{T}_{\text{type}}, L_T, F, B)$ {structured DSL: types, depths, fusion F , bridge flag B }
- 11: **Fusion:** $\mathbf{H}_{\text{out}} = \tilde{\mathbf{H}}_{\text{sp}} + \tilde{\mathbf{H}}_{\text{tm}}$ or $\{\mathbf{H}_{\text{cat}} = [\tilde{\mathbf{H}}_{\text{sp}}; \tilde{\mathbf{H}}_{\text{tm}}], \mathbf{H}_{\text{out}} = \sigma(\mathbf{W}_c \mathbf{H}_{\text{cat}} + \mathbf{b}_c)\}$ {addition vs. concatenation}
- 12: **Stage 3: Code Agent (Formal Compilation)**
- 13: $\Pi \leftarrow \text{Parse}(\mathcal{S}_{\text{DSL}})$ {blueprint (modules, I/O, order)}
- 14: $\Omega \leftarrow \text{Retrieve}(\Pi)$ {class templates, forward, interface specs}
- 15: $C \leftarrow \text{Assemble}(\Omega)$ {STModel with `__init__()`, `forward()`, `metadata/logs`}
- 16: **return** C

E Agent Collaboration Mechanism and Mutation Strategy Principles

The core mechanism of ST-LEGO is formed by the collaboration of three intelligent agents: Prompt Agent, Assemble Agent, and Code Agent. Together, they establish a closed-loop automated modeling process that spans from language understanding to structural generation and code synthesis. This process is enabled through modular design and the intermediate structure representation DSL, facilitating multi-stage decoupling that allows spatio-temporal model design to achieve high flexibility and structural controllability.

E.1 Multi-Agent Collaboration

The overall architecture of ST-LEGO draws on the concept of Multi-Agent Systems (MAS), decomposing the modeling process into cooperative subtasks. Each agent is assigned a clear role with well-defined interfaces and communicates through intermediate representations. This paradigm has been widely adopted in task-intensive domains such as control, planning, and decision-making, demonstrating robust scalability and generalization across tasks [47].

Recent works have also applied LLMs to program generation and structural planning. For instance, Code-as-Policies [5] employs LLMs to generate control logic, while Gorilla [18] uses natural language to orchestrate large-scale API calls. These methods showcase the structural reasoning capabilities of LLMs and validate the feasibility of the “language + module” paradigm, offering theoretical inspiration for ST-LEGO’s agent-based collaborative design.

E.2 Structural Mutation Strategies

The two mutation strategies in the Assemble Agent—layer fine-tuning and distribution-guided mutation—are designed to balance stability and diversity in structure generation. They serve as key mechanisms for controllable exploration of the architectural space.

In the layer fine-tuning strategy, we apply a ± 1 perturbation to the depth L of spatial or temporal modules:

$$L' = \max(1, \min(L_{\max}, L + \delta)), \quad \delta \in \{-1, 1\} \quad (18)$$

This strategy is motivated by the principle that local search around a fixed structural backbone can fine-tune model capacity and convergence behavior without compromising expressiveness. The idea is inspired by local perturbation strategies in neural architecture search (NAS) such as DARTS and ENAS [32].

In the distribution-guided mutation strategy, we collect component usage frequencies from historical models in benchmark datasets to form a task-conditioned probability distribution $\mathcal{P}(M | D)$, which guides module selection:

$$M' \sim \mathcal{P}(M | D) \quad (19)$$

This strategy is influenced by the use of performance priors to steer structural evolution, akin to guided sampling mechanisms in frameworks like AutoFormer and TE-NAS [8]. Unlike prior work, however, we integrate this strategy within an LLM-driven modular composition system, combining language-based flexibility with structure-aware performance constraints to ensure both accuracy and adaptivity.

The structural mutation mechanisms in ST-LEGO are restructured and integrated based on existing theories in architecture search and representation learning. They represent an innovative

path toward controllable and semantically aligned structure generation under the LLM-driven modeling paradigm.

F Datasets Statistics

The datasets used in the experiments include the METR-LA dataset, collected from loop detectors on Los Angeles freeways, and five benchmark datasets (PeMS-Bay, PeMS03, PeMS04, PeMS07, and PeMS08) collected by the California Department of Transportation Performance Measurement System (PeMS) [21, 34]. All datasets have a unified recording interval of 5 minutes. These datasets cover traffic information across different temporal and spatial ranges. Detailed statistics of these datasets are provided in Table 6.

Table 6: Statistics of all datasets.

Dataset	METR-LA	PeMS-Bay	PeMS03
Start Time	2012/3/1	2017/1/1	2018/9/1
End Time	2012/6/30	2017/5/31	2018/11/30
Time Interval	5 min	5 min	5 min
# Object	207	325	358
# Timestamps	34272	52116	26208
Dataset	PeMS04	PeMS07	PeMS08
Start Time	2018/1/1	2017/5/1	2016/7/1
End Time	2018/2/28	2017/8/31	2016/8/31
Time Interval	5 min	5 min	5 min
# Object	307	883	170
# Timestamps	16992	28224	17856

G Accuracy Metric Formulas

Mean Absolute Error (MAE): Measures the average absolute difference between the predicted values and the actual values.

$$MAE = \frac{1}{T_p} \sum_{t=1}^{T_p} \frac{1}{N} \sum_{i=1}^N |y_t^{r_i} - \hat{y}_t^{r_i}| \quad (20)$$

Root Mean Squared Error (RMSE): Reflects the square root of the average squared differences between predicted values and actual values, emphasizing the impact of larger errors.

$$RMSE = \sqrt{\frac{1}{T_p} \sum_{t=1}^{T_p} \frac{1}{N} \sum_{i=1}^N (y_t^{r_i} - \hat{y}_t^{r_i})^2} \quad (21)$$

Mean Absolute Percentage Error (MAPE): Measures the percentage difference between the predicted and actual values, reflecting the size of the relative error.

$$MAPE = \frac{1}{T_p} \sum_{t=1}^{T_p} \frac{1}{N} \sum_{i=1}^N \left| \frac{y_t^{r_i} - \hat{y}_t^{r_i}}{y_t^{r_i}} \right| \quad (22)$$

H Baseline Descriptions

To systematically evaluate the structural assembly and forecasting performance of ST-LEGO, we compare it with 18 representative spatio-temporal models, spanning GNNs, Transformer-based architectures, and efficient approaches. Except for ASTGCN, STSGCN, STGODE, STAEformer, MegaCRN, and TimeMixer from official repositories, the others are reproduced from public spatio-temporal benchmarks² [37]. Due to space limits, detailed descriptions are provided in the anonymous repository.

²<https://github.com/GestaltCogTeam/BasicTS>